Tutorial #5-Mars Lander

Introduction

Welcome to the fifth of our AppGameKit Studio tutorials, as before, these are aimed at those of you who are just starting to explore the world of tier 1 (BASIC) coding.

We previously looked at a number of different games, and once again, we will be expanding on the lessons learned from those as well as exploring some different techniques.

For this tutorial, we'll be creating another game, where the player controls a falling lander through the use of thrusters. As before, you will be guided through each of the steps clearly, and be encouraged to change various values and lines of code so that you can learn first-hand how simple changes can affect the code.

The entry level for these tutorials expects some initial understanding of the students in the use of AppGameKit Studio. For more beginner level tutorials they can read the AppGameKit Studio Online Guides and watch the YouTube Videos.

Students are also encouraged to join the Official AppGameKit Studio forums. There's a newbie section where new users will be helped by many experienced and friendly developers who use AppGameKit Studio all the time.

Objectives

These tutorials aim to teach coding and game development using a combination of examples and action-based teaching. You will learn from studying the code snippets, reading and understanding the comments and gain an additional understanding of each section through a number of simple exercises aimed at reinforcing each lesson and will help build student confidence by demonstrating the ease with which the code can be altered.

This fifth tutorial aims to expand on the concepts of basic coding covered by the previous tutorials and is designed to lay the groundwork of future developments. During this tutorial we will:

- Expand on previous lessons to re-enforce learning
- Create a pseudo physics environment to simulate gravity
- Learn some new techniques to fetch data on sprites

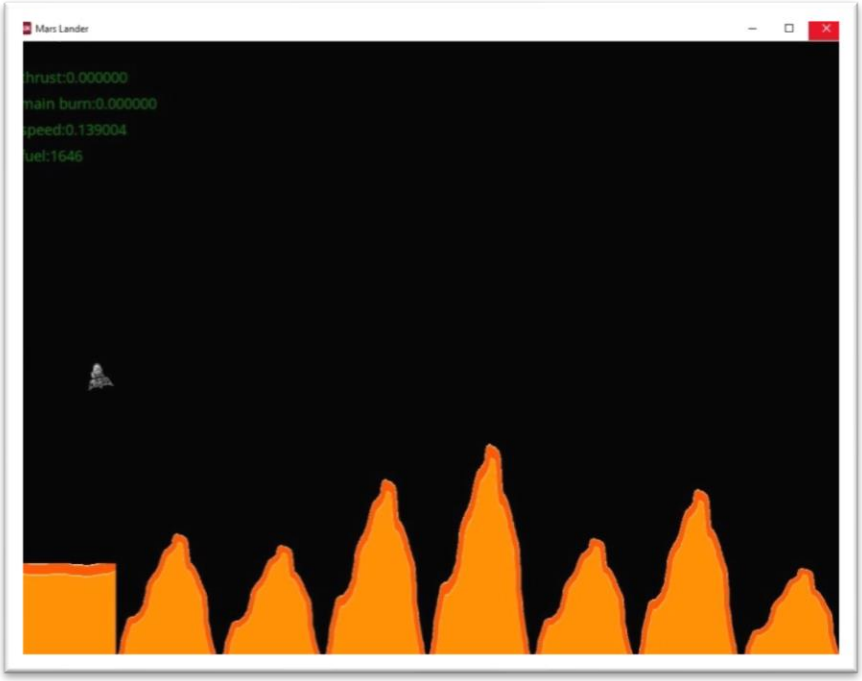
At any point, examples of almost all coding commands can be accessed from the IDE (integrated development environment) simply by either clicking directly to the left of the command and pressing F1 on the keyboard. You're encourage to learn this method from the beginning as it will prove to be an invaluable reference guide and is a resource often used even by coders who are very familiar with AppGameKit Studio.

You can also access a full command reference from the IDE. By clicking Help on the top tool bar, you can gain access to a number of reference sources, including video tutorials, a list of useful keyboard shortcuts as well as the command reference.

Each section of code is available as a separate project which can be simply loaded into the IDE, but it's recommended that you learn to type the code into the IDE, as this will help to develop a familiarity with AppGameKit Studio overall and will promote confidence in using the functions of the IDE as well as reinforcing the learning process.

So, what game are we making?

For this tutorial, we are aiming to combine lots of the techniques we learnt in our previous tutorials. You'll notice a lot of familiar code, and techniques. This time, we are making a game called Mars Lander, where the player controls a Mars rover landing on the red planet.



Many of the coding practices we will cover in this game will be familiar to you and this tutorial aims to re-enforce everything you've learnt to date. We will of course take a look at the fully documented code in detail and provide a number of exercises to expand your understanding.

Due to the nature of these kinds of games, the code itself is fairly short, so we'll provide this as a single block, with explanations.

As always, we set up the project and define the screen size.

```
// Project: Mars Lander
// Created: 2017-06-20

// show all errors

SetErrorMode(2)

// set window properties
SetWindowTitle( "Mars Lander" )
SetWindowSize( 1024, 768, 0 )
SetWindowAllowResize( 1 ) // allow the user to resize the window

// set display properties
SetVirtualResolution( 1024, 768 ) // doesn't have to match the window
SetOrientationAllowed( 1, 1, 1, 1 ) // allow both portrait and landscape on mobile devices
SetSyncRate( 60, 0 )
SetScissor( 0,0,0,0 ) // use the maximum available screen space, no black borders
UseNewDefaultFonts( 1 ) // since version 2.0.22 we can use nicer default fonts
```

Now let's create our types, load in some images and create a few sprites. As you have done this several times before, we won't embellish on this, but do encourage you to read through the code to refresh your memory if needed.

```
//load 2 images for the mars scape
global landscape1=1
global landscape2=2

LoadImage(landscape1,"surface1.png")
LoadImage(landscape2,"surface2.png")

//create a sprite for rockets

global flame=5
global flame2=6
loadimage(flame,"flame.png")
createsprite(flame,flame)
//hide it off screen for now
SetSpritePosition(flame,-1000,-1000)
//side thrust
createsprite(flame2,flame)
//hide it off screen for now
SetSpritePosition(flame2,-1000,-1000)

//set up a new type for the mars scape
type scapes
        x as integer
        y as integer
        sprite as integer
endtype

//set up an array
dim landscape[15] as scapes

//make some sprites
for makesprites=1 to 15
        landscape[makesprites].sprite=9+makesprites
        createsprite(landscape[makesprites].sprite,landscape1)
        SetSpritePosition(landscape[makesprites].sprite,-100000,-100000)
next makesprites

//set up some timer variables
global frameadjust as float
global currenttime as integer

//set up some variables to hold win and lose conditions
global landed as integer
global landedtime as integer


//create a type for the lander

type marslander
        x as float
        y as float
        angle as float
        sprite as integer
        velocity as float
        reverse as float
        currentthrust as float
        fuel as integer
endtype

//make the global variable

Global lander as marslander

// create a sprite
lander.sprite=4
Loadimage(lander.sprite,"lander.png")
CreateSprite(lander.sprite,lander.sprite)
//give it polygon collision
SetSpriteShape(lander.sprite,3)
//hide the sprite off screen
SetSpritePosition(lander.sprite,-1000,-1000)
```

```
//let's make some debris

For makedebris=1 to 5
        d$="debris"+str(makedebris)+".png"
        LoadImage(29+makedebris,d$)
        CreateSprite(29+makedebris,29+makedebris)
        SetSpritePosition(29+makedebris,-10000,-10000)
Next makedebris

//create some text objects to hold information
//on lander stats

Createtext(1,"thrust")
//set the text size
SetTextSize(1,24)
//we'll make all of our text green
SetTextColor(1,0,127,0,255)
SetTextPosition(1,10,32)
Createtext(2,"burn")
SetTextSize(2,24)
SetTextColor(2,0,127,0,255)
SetTextPosition(2,10,64)
Createtext(3,"speed")
SetTextSize(3,24)
SetTextColor(3,0,127,0,255)
SetTextPosition(3,10,96)
CreateText(4,"fuel")
SetTextSize(4,24)
SetTextColor(4,0,127,0,255)
SetTextPosition(4,10,128)

//we'll also need a text object to report winning and failure

CreateText(5,"")
SetTextSize(5,64)
SetTextColor(5,0,127,0,255)
SetTextPosition(5,300,300)

//hide it, as we only want to show if it something happen

SetTextVisible(5,0)

//Lastly we'll need a couple for the main menu

CreateText(6,"MARS LANDER")
SetTextSize(6,64)
SetTextColor(6,0,127,0,255)
SetTextPosition(6,350,200)

CreateText(7,"PRESS SPACE")
SetTextSize(7,64)
SetTextColor(7,0,127,0,255)
SetTextPosition(7,370,280)

//lastly we'll load a couple of sounds

Global thrustsound:thrustsound=LoadSound("thrust.wav")
Global thrustplaying=0
Global bangsound:bangsound=LoadSound("bang.wav")

//set a label so we can jump back to this point
//in the code
```

We will create a label, so we can jump back to this part of the code using the GOTO command. Some purists don't approve of the use of GOTO in structured code, but it can be a very quick way of repeating a loop. Whether you decide to use GOTO (or indeed GOSUB, which will jump to a section of your code, but allow you to return from it in a similar way to a function), is up to you, and you will develop your own coding style and preferred methods as you learn more about coding.

```
_restart:

//call the main menu
mainmenu()

do
```

```
        //we'll use timer based movement
        //for smooth gameplay on lower end machines
        frameadjust = 60*GetFrameTime()
        currenttime=GetMilliseconds()
```

We will only call the main functions to control the game if the lander hasn't landed or crashed. This will prevent the player from continuing the game after it's over.

```
        if landed=0
                //check collisions
                collisions()
                //apply psuedo phyics
                adjustlander()
                //control lander
                controllander()
                //draw the lander
                drawlander()

                //show updates
                stats()
        endif

        //if the game has been won or lost
        //and more than 3 seconds has passed
        //return to main menu

        if landed<>0 and currenttime-landedtime>3000
                goto _restart
        endif

    Sync()
loop

Function drawlander()

        //set lander angle
        SetSpriteAngle(lander.sprite,lander.angle)

        //set lander position
        SetSpritePosition(lander.sprite,lander.x,lander.y)

        //if the lander has thrust applied display the jet sprite
        if lander.reverse>0
                //make it visible
                SetSpriteVisible(flame,1)
                //we randomly change the sprites size to give
                //the impression of flamming
                setspritesize(flame,random(8,14),random(18,24))
                //set the sprite position
                SetSpritePosition(flame,lander.x+12,lander.y+32)
                else
                //if not thrusting
                //hide the sprite
                SetSpriteVisible(flame,0)
        endif

        //show side thrusters if active
        if lander.currentthrust>0.002
                //make it visible
                SetSpriteVisible(flame2,1)
                SetSpriteAngle(flame2,90)
                //we randomly change the sprites size to give
                //the impression of flamming
                setspritesize(flame2,random(8,14),random(18,24))
                //set the sprite position
                SetSpritePosition(flame2,lander.x-8,lander.y+4)
        endif

        //show side thrusters if active
        if lander.currentthrust<0.002
                //make it visible
                SetSpriteVisible(flame2,1)
                SetSpriteAngle(flame2,270)
                //we randomly change the sprites size to give
                //the impression of flamming
                setspritesize(flame2,random(8,14),random(18,24))
                //set the sprite position
                SetSpritePosition(flame2,lander.x+26,lander.y+4)
```

```
        endif

        //hide side thrust if near 0
        if abs(lander.currentthrust)<=0.001
        SetSpriteVisible(flame2,0)
        endif

EndFunction
```

We use the arrow keys to apply thrust to our lander. No direct movement takes place in the controllander function(), we simply alter the amount of thrust we need to apply to the lander, which is then applied in the adjustlander() function.

```
Function controllander()

        //Left burn
        If GetRawKeyState(39) and lander.fuel>0
                //increase left thrust
                inc lander.currentthrust,.004
                //decrease fuel
                dec lander.fuel,1
                //if not playing the thrust sound, play it now
                If GetSoundInstancePlaying(thrustplaying)=0
                thrustplaying=playsound(thrustsound)
                Endif
        Endif

        //Right burn
        If GetRawKeyState(37) and lander.fuel>0
                //increase right thrust
                dec lander.currentthrust,.004
                //decrease fult
                dec lander.fuel,1
                //if not playing the thrust sound, play it now
                If GetSoundInstancePlaying(thrustplaying)=0
                thrustplaying=playsound(thrustsound)
                Endif
        Endif

        //Reverse
        If GetRawKeyState(38) and lander.fuel>0
                //add reverse thrust
                inc lander.reverse,.002
                //decrease fuel
                dec lander.fuel,1
                //if not playing the thrust sound, play it now
                If GetSoundInstancePlaying(thrustplaying)=0
                thrustplaying=playsound(thrustsound)
                Endif
        Endif

Endfunction
```

EXERCISE: *Alter the control keys to use, W to thrust, A to tilt right, and D to tilt left.*

EXERCISE: *Make the game more challenging by adjusting the amount of fuel used.*

The adjustlander() function is where all the real magic takes place. Here we create a 'fake' physics using the values taken from the left and right thrusters and the main burn thruster which slows decent. Too much main burn and you can leave the atmosphere, not enough and you'll crash into the surface.

```
Function adjustlander()

        //if not landed
        if landed=0
        //natural fall
        inc lander.velocity,.002

        // reverse thrust reduces speed
        dec lander.velocity,lander.reverse

        //set a maximum velocity
```

```
        if lander.velocity<-3
                lander.velocity=-3
        endif


        //if the lander has reverse thrust apply it.
        if lander.reverse>0
                dec lander.reverse,.001
                if lander.reverse<0
                        lander.reverse=0
                endif
        endif

        //if the lander has left thrust, apply it
        if lander.currentthrust>0
                dec lander.currentthrust,.001
        endif

        //if the lander has right thrust, apply it
        if lander.currentthrust<0
                inc lander.currentthrust,.001
        endif

        //adjust the angle of the lander
        inc lander.angle,lander.currentthrust
```

To keep the game simple, we won't allow the lander to tilt by more than 45 degrees in either direction.

```
        //don't let the angle become less than -45
        if lander.angle<-45
                lander.angle=-45
        endif

        //don't let the angle become greater than 45
        if lander.angle>45
                lander.angle=45
        endif

        //move the lander based on it's angle
        //and velocity
```

To ensure the lander moves in the direction it's facing, we will use a little maths. Note how we multiply our result by the value contained in *frameadjust* to ensure a consistent movement speed on all systems.

```
        movex#=sin(lander.angle)*frameadjust
        movey#=(cos(lander.angle)*lander.velocity)*frameadjust

        lander.x=lander.x+movex#
        lander.y=lander.y+movey#+lander.velocity

        endif

        //left atmosphere
        if lander.y<-64 and landed=0
                lander.y=-64
                landed=1
                landedtime=currenttime
        endif

EndFunction

Function collisions()

        //check all of the mars scapes for collision
        //with the lander
        for checkcollisions=1 to 15
                if GetSpriteCollision(lander.sprite,landscape[checkcollisions].sprite)
                whatisit=GetSpriteImageID(landscape[checkcollisions].sprite)
                //if a collision has taken place
                //the lander will crash if
                //it doesn't hit a landing (flat terrain) zone
                //if it's angle is too sharp
                //or if it is moving to quickly
                if whatisit<>landscape2 or lander.velocity>0.3 or lander.angle<-6 or
lander.angle>6 and landed=0
```

```
                //flag it as crashed
                landed=1
                //flag the time
                landedtime=currenttime
                //if it's crashed we'll
                //replace the lander with debris sprites.
                //at the lander old position
                //with some randomness)
                For placedebris=30 to 34

        SetSpritePosition(placedebris,lander.x+random(1,10),lander.y+random(1,10))
                //use physics to blow it up
                SetSpritePhysicsOn(placedebris,2)
                Next placedebris

                //hide the lander off screen
                lander.x=-10000
                lander.y=-10000
                SetSpritePosition(lander.sprite,lander.x,lander.y)

                //play the crash sound
                PlaySound(bangsound)
                else
                //otherwise, it will land
                //successfully
                //so we set the landed varible to indicate this.
                landed=2
                //flag the time
                landedtime=currenttime
                endif


                endif
        next checkcollisions

EndFunction

Function makemars()
        //set up a label so we can
        //repeat the loop if no landing zone made

        _repeat:

        //landscapes start at the edge of the screen
        startx=0
        //which landscape shall we start with
        nextlandscape=0

        //has a landing zone been created
        landingzone=0

        //start a loop
        Do

        //landscape image
        use=landscape1
        //use the next landscape
        inc nextlandscape,1
        //make random size
        height=random(128,356)
        width=random(128,128)

        //50% chance of this landscape being a landing zone
        //and only allow 1
        if random(1,100)>50 and landingzone=0
                //flag as created
                landingzone=1
                use=landscape2
                //set the height and width to 128
                height=128
                width=128
        endif

        //set the sprite image
        SetSpriteImage(landscape[nextlandscape].sprite,use)
        //change the sprite size
        SetSpriteSize(landscape[nextlandscape].sprite,width,height)
        //set the sprite position
        SetSpritePosition(landscape[nextlandscape].sprite,startx,768-height)
        SetSpriteShape(landscape[nextlandscape].sprite,3)
```

```
        //we will need to set physics so debris will bounce off
        SetSpritePhysicsOn(landscape[nextlandscape].sprite,1)
        //increase the screen position by the width
        //so each landscape piece is placed next to the last one
        startx=startx+width

        //if the next piece will appear off screen we've finished creating
        if startx>1024
         exit
        endif

        //keep going in needed
        Loop

        //if no landing zone has been created, try again.
        If landingzone=0 then goto _repeat


EndFunction
```

EXERCISE: *Vary the randomness of the landscape by altering the width and height of the terrain to a new random figure. Use higher numbers, or a greater range for increased difficulty.*

```
Function stats()

        //set some string variables to display
        //current stats
        thrust$="thrust:"+str(lander.currentthrust)
        burn$="main burn:"+str(lander.reverse)
        speed$="speed:"+str(lander.velocity)
        fuel$="fuel:"+str(lander.fuel)

        //set the strings
        SetTextString(1,thrust$)
        SetTextString(2,burn$)
        SetTextString(3,speed$)
        SetTextString(4,fuel$)

        //set win / lose text
        //if needed
        if landed=1
                SetTextString(5,"Landing Failed")
                SetTextVisible(5,1)
        endif

        if landed=2
                SetTextString(5,"Landing Success")
                SetTextVisible(5,1)
        endif

EndFunction
```

EXERCISE: *Create a new text object and display it to show the current angle of the lander. Remember that this is stored in the variable lander.angle.*

```
Function mainmenu()

//set the main variables

//set a random start
lander.x=random(300,700)
lander.y=20

lander.velocity=.001
lander.fuel=2000

//set the lander position
SetSpritePosition(lander.sprite,lander.x,lander.y)

//make mars landscape
makemars()

//reset win/lose

landed=0

//hide win/lose text
SetTextVisible(5,0)
//show menu text
SetTextVisible(6,1)
SetTextVisible(7,1)

//hide debris
For hide=30 to 34
        SetSpritePosition(hide,-1000,-1000)
Next hide

//wait until space is pressed
do

        If GetRawKeyState(32)
         exit
        Endif

        Sync()
loop

//hide menu text
SetTextVisible(6,0)
SetTextVisible(7,0)

EndFunction
```